Cybersecurity Model Review — Plain-English Report

Oct 14, 2026 · @Mussab

The short answer

Use the Original XGBoost. In a fair re-test on cleaned data it scored the highest overall (99.73% accuracy, 0.867 average score across attack types) and raised about 3 times fewer false alarms than either weighted model (112 vs 309 and 324). The two weighted models catch slightly more attacks (missing 82–86 instead of 140, out of 54,000). Choose Standard Weighted XGBoost only if a missed attack costs far more than a false alarm. The Targeted version adds nothing measurable over Standard, so it is not worth the extra complexity.

One caution applies to all three: when tested on attack types they never saw in training, they caught only 24–29% of them.

What the project does

The project teaches a computer to look at a network connection and say whether it is normal or an attack, and if it is an attack, which kind.

Think of a security guard at a building entrance. The guard has watched thousands of past visitors, each labelled "fine" or "trouble — and what kind". After enough examples, the guard learns the signs: someone trying 50 doors in a minute is probably scanning the building; someone flooding the front desk with fake requests is trying to jam it. The model is that guard, and the past visitors are the dataset.

The project ran in 10 phases: understand the data, clean it, balance it, train many candidate models, explain their decisions, simulate a response (alert / allow / review), then test and improve the best one.

The data, and what the review found

The data is NSL-KDD, a well-known practice dataset of 148,517 network connections, each described by 41 measurements and labelled "normal" or one of 39 attack types. Two numbers say most of it: 53% of rows are normal traffic and 32% are a single attack (neptune, a flooding attack). Many attack types have fewer than 20 examples, and 6 have fewer than 5.

Three things in the data matter for trusting the results:

- **The same connection appears more than once.** 15,791 rows are exact copies of another row. When the project mixed the two files and split them again, 5,086 test rows (17.1%) ended up as copies of rows the model had already studied. That is like giving a student the exam questions as homework: the score goes up but tells you less.
- **A few rows contradict each other.** 30 identical-looking connections carry different labels. No model can get all of those right.
- **One column is always zero** (`num_outbound_cmds`). It carries no information and was safely dropped.

The data has no missing values, and no column secretly reveals the answer — both were checked.

How the data was prepared

Before a model can learn, the raw data has to be tidied. The project did this in six steps:

1. **Merged the two files and re-split them 80/20.** The original test file contains 17 attack types that never appear in the training file. Merging means every attack type is seen during training. The trade-off: the model is no longer tested on surprise attacks (see the results section).
2. **Dropped 9 attack types with fewer than 10 examples** (38 rows). Too few to learn from or to test fairly. 28 categories remain: normal + 27 attacks.
3. **Turned words into numbers.** Columns like protocol ("tcp", "udp") became codes (0, 1, 2), because models only read numbers. The codes were learned from the training part only — correct practice.
4. **Put all measurements on the same scale** (scaling). Needed for some models, harmless for the tree-based ones.
5. **Balanced the classes.** Two methods were tried. *Oversampling* copies examples of rare attacks until each has at least 1,000. *Class weighting* keeps the data as is but tells the model "a mistake on a rare attack costs more".
6. **Saved everything** so later phases reuse the same prepared data.

Steps 2–6 are sound. The one gap is that duplicates were checked in each file separately but not after merging, which caused the 17% overlap described above. The corrected version removes them right after merging.

The three models

In Phase 4 the project tried 11 kinds of model (logistic regression, decision tree, random forest, SVM, KNN, naive Bayes, XGBoost, LightGBM, gradient boosting, AdaBoost, and a neural network). XGBoost was chosen: it was the most accurate and trained in about 36 seconds, versus 34 minutes for the close runner-up.

XGBoost works like a committee of 200 simple flowcharts ("is the connection count above 300? is the error rate above 0.8?"). Each new flowchart focuses on the cases the earlier ones got wrong, and the committee votes.

The three final models are the same XGBoost committee, trained three ways to cope with rare attacks:

Model	How it handles rare attacks	Everyday analogy
Original XGBoost	Copies rare-attack examples until each type has 1,000	Showing a student the same few flashcards many times
Standard Weighted XGBoost	Keeps the data as is; a mistake on a rare type counts as much more costly, in proportion to its rarity	Telling the student "these few questions are worth more marks"
Targeted Weighted XGBoost	Same as Standard, plus extra weight (2–5×) on 4 attacks it kept missing: rootkit, multihop, ftp_write, loadmodule	Extra marks on the four questions the student failed last time

How the models were judged, and the results

All three models were re-trained on identical cleaned data and tested 5 times, each time on a different fifth of the data the model had not seen (cross-validation). Every connection was tested exactly once. Two kinds of mistake matter most:

- **Missed attack** — an attack let through as normal. The dangerous mistake.
- **False alarm** — normal traffic flagged as an attack. Wastes analysts' time; too many and people stop trusting alerts.

Original raises ~3× fewer false alarms; weighted models miss ~40% fewer attacks

Counts over 132,688 connections, 5-fold cross-validation (lower is better)



results/fair_comparison/cv_summary.csv · src/evaluation/compare_models.py, run 2026-10-14

The overall scores are very close:

Model	Accuracy	Average score across attack types (macro F1)	Wrong labels	Official test: attacks caught	Official test: new attack types caught
Original XGBoost	99.73%	0.867	355	86.3%	23.7%
Standard Weighted XGBoost	99.62%	0.859	504	86.8%	28.0%
Targeted Weighted XGBoost	99.61%	0.858	515	86.9%	29.4%

The average-score gap (0.867 vs 0.858) is smaller than how much each model's score moves from one test round to the next (about ±0.014), so no model clearly wins on that measure alone. The clear differences are false alarms (Original is best) and missed attacks (weighted is best).

The "official test" columns use the dataset as its authors intended: train on the training file, test on the test file, which holds 1,146 attacks of types never seen in training. All three catch about 93% of known attack types but only 24–29% of new ones. That is the honest measure of how a model would cope with a brand-new attack.

Rare attacks remain hard for every model. Rootkit (13 examples) was caught 2–3 times out of 13, and multihop (15 examples) 3–5 times out of 15. The Targeted model's extra weight on these did not reliably help.

Problems found and what was changed

The code is careful overall: encoders and scalers learn from training data only, later phases use a separate validation set, and the reports state their limits honestly. The problems below are the ones that change how far the results can be trusted.

Problem	Why it matters	What was done
17.1% of test rows are copies of training rows	Scores look better than they really are	New `src/preprocessing/build_dataset.py` removes duplicates after merging
The test set was used to pick and then tune models (Phases 4, 8, 10)	The test stops being a fair final exam	New `src/evaluation/compare_models.py` re-tests all three with 5-fold cross-validation
The targeted weights were chosen after looking at test-set mistakes on 2–3 examples	The "improvement" may only fit those few rows	Re-tested fairly, see results
Phase 10.17 ranks models after leaving out the two attacks every model fails (rootkit, multihop)	Leaving them out is what puts Targeted on top (0.903 vs 0.901)	Fair re-test uses every class
The Original model trained on 100% of training data, the weighted ones on 80%	Not a like-for-like comparison	All three now train on identical data
Phases 5–10 pointed to `D:\smart-cybersecurity-ml`, a folder that does not exist in this repo	Those notebooks crash when run here	Changed to paths relative to the project (6 notebooks)
Phase 4 charts and tables used typed-in numbers; one cell used a folder variable before creating it	Re-running gives stale charts or an error	Now read from the computed results

Problem	Why it matters	What was done
LightGBM (52% accuracy) and AdaBoost (6.6% macro F1) results look like setup problems, not real model limits	They were dropped too early	Noted; not re-run

Other notes: `api/main.py` and the `src/` sub-folders were empty before this review, and about 150 MB of data, models and zip files are stored in git, which makes the repository slow to clone.

Recommendations and next steps

1. **Keep the Original XGBoost as the main model.** It is the simplest of the three and gives the fewest false alarms, with the best overall score.
2. **Decide how much a missed attack costs compared with a false alarm.** If missing one attack is worth more than about 4 extra false alarms, switch to Standard Weighted XGBoost (54 fewer missed attacks for 197 more false alarms). Drop the Targeted version either way.
3. **Report the cleaned, cross-validated numbers** (99.73% accuracy, 0.867 macro F1), not the earlier ones measured on the overlapping test set.
4. **Quote the new-attack result alongside the headline.** About 1 in 4 never-seen attack types is caught. To improve this, add an "unusual traffic" detector (anomaly detection) next to the classifier, and test on a second dataset such as CICIDS2017.
5. **Group the rarest attacks.** Rootkit, multihop and loadmodule have 10–15 examples each, too few to learn. Merging them into their family ("user-to-root" attacks) would give the model enough examples. After duplicates are removed, ftp_write has only 9 rows and falls below the 10-example cutoff.
6. **Re-run the notebooks from Phase 2** with the duplicate fix, so the saved models and charts match the cleaned data. The fair comparison can be re-run any time with `python -m src.evaluation.compare_models` (about 45 minutes on this machine).